

Loan System Test Report

Date: 2026-03-15

Project: Loan System

Workspace: C:\Users\ngoga\Desktop\Loan system

Purpose

This report records the automated tests added for the current Loan System codebase, what they cover, how they were executed, and the remaining testing gaps before deployment.

Test Scope

Automated tests were added for the most important logic recently changed in the system:

- frontend authentication and session handling
- frontend loan term auto-calculation
- frontend default creditline generation
- backend application creation logic
- backend application payment plan resolution
- backend application update logic
- backend login and auth route behavior

These tests are mainly unit and route-level integration tests. They do not yet include full browser end-to-end tests.

Frontend Tests

Frontend test runner:

- command: `npm test`
- location: `loan-system-frontend`
- implementation file: `loan-system-frontend/tests/run-tests.mjs`

Frontend Files Covered

- `loan-system-frontend/src/auth/auth.js`
- `loan-system-frontend/src/utils/application.js`

Frontend Test Cases

1. `getTokenExpiry` returns expiry in milliseconds from a valid JWT payload.
2. `isTokenExpired` treats malformed tokens as expired.
3. `saveAuth` and `getAuth` correctly persist and restore valid auth state.
4. Expired tokens are cleared from storage and treated as logged out.

5. `clearAuth` removes all stored authentication fields.
6. `calculateTermMonths` returns the ceiling of `amount / payment_plan`.
7. `calculateTermMonths` returns an empty value for invalid input.
8. `buildDefaultCreditline` generates a value from client account when available.
9. `buildDefaultCreditline` falls back to client ID when account is missing.

Frontend Result: Passed

Backend Tests

Backend test runner:

- command: `python -m unittest discover -s tests -v`
- location: `loan_system_backend`
- test files: `loan_system_backend/tests/test_applications.py` ,
`loan_system_backend/tests/test_auth.py`

The backend tests were executed with the backend virtualenv packages available through `PYTHONPATH` , because the base Python environment did not include Sanic.

Backend Files Covered

- `loan_system_backend/app/routes/applications.py`
- `loan_system_backend/app/routes/auth.py`

Backend Test Cases

1. `aggregate_creditlines` correctly sums and selects aggregate values.
2. `_resolved_payment_plan` prioritizes application-level payment plan first.
3. `create_application` uses an available client creditline when present.
4. `create_application` persists `payment_plan` on the new application.
5. `get_application` returns the resolved application payment plan.
6. `update_application` updates `amount_requested` .
7. `update_application` updates `payment_plan` .
8. `update_application` updates `purpose` .
9. `update_application` updates `term_requested` .
10. `login` returns token, role, and user ID for valid credentials.
11. `login` rejects invalid credentials with `401` .

Backend Result: Passed

Verification Performed

- frontend automated tests passed
- backend automated tests passed
- backend Python syntax check passed
- frontend production build passed

Frontend production build command: `npm run build`

Build note:

- Vite build completed successfully
- large chunk warning still exists for the frontend bundle and should be handled later as a performance improvement, not as a functional blocker

Current Coverage Summary

Covered well:

- auth storage and session-expiry logic
- term calculation logic
- default creditline generation
- payment plan persistence path in backend
- application update behavior
- login success and failure behavior

Not covered yet:

- full browser UI interaction tests
- live database integration tests against PostgreSQL
- full create-to-score-to-decision end-to-end workflow
- role-based navigation rendering in the browser
- deployment environment smoke tests

Risks And Gaps

1. No real browser automation yet.
2. Backend tests use mocked async session behavior, so they validate route logic but not actual PostgreSQL query execution.
3. No load, security, or concurrency testing has been added.
4. No production deployment smoke test has been executed yet.

Recommendation

The current test suite is a good baseline for protecting the core logic that was recently changed. Before deployment, the next layer should be:

1. deployment checklist
2. database migration and backup verification
3. manual smoke test in a staging or production-like environment
4. optional browser E2E tests for login, create application, edit application, and view application details